STA410 Assignment 3

Talia Fabregas 1008350213

April 3, 2026

Code: https://github.com/taliafabs/sta410-a3.git

1. **Randomized SVD** (full disclosure: I used ChatGPT to get the commands to load the dataset)

   (a) Centering and scaling the columns of X so that they have mean 0 and standard deviation 1, using `numpy` to normalize the data.

```python
import numpy as np
import statsmodels as sm
from statsmodels import datasets

# random number generator
rng = np.random.default_rng(410)

# load the dataset (un-normalized)
X_data = sm.datasets.get_rdataset("mtcars").data.values
n, p = X_data.shape[0], X_data.shape[1]

# normalize the data so each col has mean=0, std=1
X_normalized = (X_data - np.mean(X_data, axis=0)) / np.std(X_data, axis=0)
```

   My code chunk above loads necessary packages, defines an rng, loads the un-normalized dataset X, and uses the numpy python library to center and scale the columns of X to give them mean 0 and standard deviation 1

   (b) How centering affects the condition number & how this relates to the spread of the singular values

```python
# Condition numbers
unnormalized_cond_num = np.linalg.cond(X_data)
normalized_cond_num = np.linalg.cond(X_normalized)

# Singular values
singular_values = np.linalg.svd(X_data, compute_uv=False)
singular_values_normalized = np.linalg.svd(X_normalized, compute_uv=False)

# QR
Q_uncentered, R_uncentered = np.linalg.qr(X_data)
Q_centered, R_centered = np.linalg.qr(X_normalized)

# Compare the condition numbers of X before & after normalization
print(f"Condition numbers of the matrix X:")
print(f"Before normalization (centering): {unnormalized_cond_num}")
print(f"After normalization (centering): {normalized_cond_num}")

# Condition numbers of R (should be the same as X)
print(f"Condition numbers of R from the QR factorization, X=QR")
print(f"Before centering & scaling: {np.linalg.cond(R_uncentered)}")
print(f"After centering & scaling: {np.linalg.cond(R_centered)}")


# Plot the singular values on a log scale

# before centering & scaling
```

```python
plt.figure(figsize=(8, 5))
plt.semilogy(singular_values, marker='o', linestyle='-', markersize=3)
plt.title('Singular values before normalization')
plt.xlabel('Index')
plt.ylabel('Singular value magnitude (log scale)')
plt.grid(True, which="both", ls='-', alpha=0.5)
plt.show()

# after centering & scaling
plt.figure(figsize=(8, 5))
plt.semilogy(singular_values_normalized, marker='o', linestyle='-', markersize=3)
plt.title('Singular values after normalization')
plt.xlabel('Index')
plt.ylabel('Singular value magnitude (log-scale)')
plt.grid(True, which="both", ls='-', alpha=0.5)
plt.show()
```

```
Condition numbers of the matrix X:
Before normalization (centering): 1562.2158946247298
After normalization (centering): 17.314062023893836
Condition numbers of R from the QR factorization, X=QR
Before centering & scaling: 1562.2158946247298
After centering & scaling: 17.314062023893836
```

Figure 1: The condition numbers of matrix X and the condition numbers of the matrix R from the QR factorization $X = QR$, before and after centering & scaling (normalizing) the columns of X

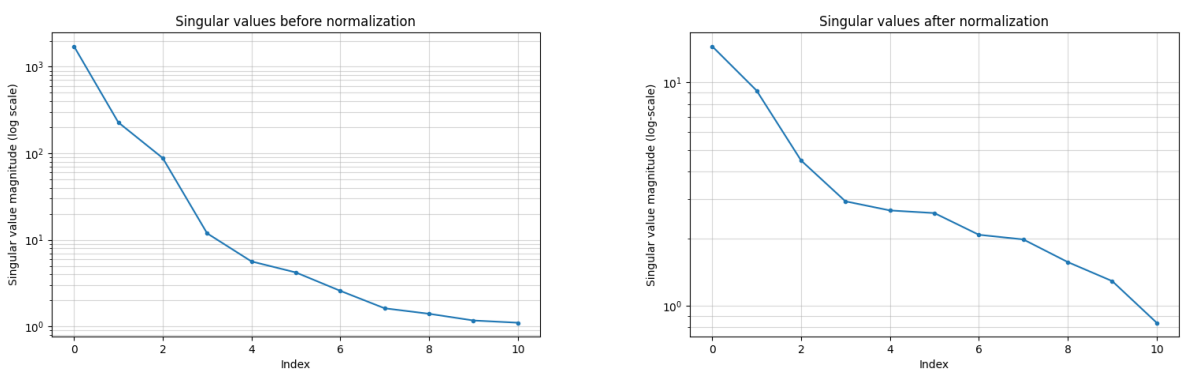

Figure 2: Singular values of the matrix X on a log scale, before and after centering

Figure 1 shows the condition numbers of X and R from the QR factorization $X = QR$ before and after centering & scaling the columns of X, which are equal since $\kappa(X) = \kappa(R)$. Before centering and scaling, $\kappa(X) = \kappa(R) \approx 1562.22$, but centering & scaling the columns of X to have mean 0 and standard deviation 1 reduced it to $\kappa(X) = \kappa(R) \approx 17.31$. In figure 2, we can see that centering & scaling the columns of X significantly reduced the spread of its singular values, which is why the condition number was reduced.

(c) Implementation of the Randomized SVD Procedure

```python
def randomized_svd(X=X_normalized, r=5):
    # step 1
    # (I initially wrote this part iteratively & asked chatgpt how to vectorize)
    P = rng.standard_normal(size=(p, r))
    # step 2
    Z = np.dot(X, P)
```

```python
    # step 3
    Q, R = np.linalg.qr(Z)
    # step 4
    Y = np.dot(Q.T, X)
    # step 5
    U_y, svs, V_y_transpose = np.linalg.svd(Y, full_matrices=False)
    D_y = np.diag(svs)
    # step 6
    return Q @ U_y @ D_y @ V_y_transpose
```

The code chunk above implements the 6 steps of the randomized svd procedure outlined on the a3 handout.

(d) Finding the smallest ratio of r to p for which

$$||X - QU_Y D_Y V_Y^\top||_1 / ||X||_1 < 0.2$$

where $||A||_1$ is the sum of the absolute values (L1-norm) for the matrix A.

```python
def find_smallest_ratio(X=X_normalized, threshold=0.2):
    """
    Find the smallest ratio of r to p for which:
        ||X - Q U_y D_y V_y^T||_1 / ||X||_1 < 0.2
    where ||A||_1 is the sum of the absolute values of matrix A.
    """
    p = X.shape[1]
    l1_norm_X = np.linalg.norm(X, ord=1)
    ratio_record=[]
    for r in range(1, p+1): # r = 1, 2, ... p
        X_r = randomized_svd(X=X, r=r)
        l1_norm_error = np.linalg.norm((X-X_r), ord=1)
        ratio = l1_norm_error / l1_norm_X
        ratio_record.append(ratio.copy())
        if ratio < threshold:
            print(f"r:{r}, p:{p}, smallest ratio of r to p:{r / p}, norm_ratio={ratio},")
            return ratio_record
    print(f"no r  p achieved l1 norm ratio < {threshold}")
    return ratio_record

print(f"Part (d) output:")
# before normalization aka centering & scaling
print(f"Without centering & scaling")
error_ratio_unnormalized = find_smallest_ratio(X=X_data)
print(error_ratio_unnormalized)
print(f"With centering & scaling")
# after normalization aka centering & scaling
error_ratio_normalized = find_smallest_ratio(X=X_normalized)
print(error_ratio_normalized)
```

```
Part (d) output:
Without centering & scaling
r:2, p:11, smallest ratio of r to p:0.18181818181818182, norm_ratio=0.17502649181841778,
[0.9677966851671169, 0.17502649181841778]
With centering & scaling
r:9, p:11, smallest ratio of r to p:0.8181818181818182, norm_ratio=0.1620851318620044,
[0.9698818420691514, 0.6463753956312465, 0.6516792514965031, 0.5270949837389974, 0.4698938726051309, 0.3465330681593
0.29796236726291525, 0.25393817701976906, 0.1620851318620044]
```

Figure 3: Before centering and scaling the matrix X, the smallest ratio of r to p to meet the < 0.2 threshold is 0.1818..., or $r = 2$, $p = 11$. After centering and scaling the columns of X, the smallest ratio is 0.8181..., or $r = 9$, $p = 11$

2. **Iterative Optimization Methods**
Consider minimizing the function:

$$f(x) = x^2 - 2\alpha x + \lambda |x|^\gamma$$

where $\lambda > 0$ and $0 < \gamma < 1$

(a) Show that f is minimized at $x = 0$ if and only if:

$$\lambda \geq \frac{2}{2-\gamma}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma} |a|^{2-\gamma}$$

$f(0) = 0^2 - 2\alpha 0 + \lambda|0|^\gamma = 0$, so if f is minimized at $x = 0$, $f(x) \geq 0, \forall x$ must hold true.

f is a symmetric function:

$$f(x) = \begin{cases} x^2 - 2\alpha x + \lambda x^\gamma & \text{if } x > 0 \\ x^2 + 2\alpha x - \lambda x^\gamma & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases}$$

$$\lim_{x\to+\infty} f(x) = \lim_{x\to-\infty} f(x) = \infty,$$

since the x^2 term dominates. additionally,

$$\lim_{x\to 0^-} f(x) = \lim_{x\to 0^+} f(x) = 0$$

Consider the case where $x > 0$:

$$f(x) = x^2 - 2\alpha x + \lambda x^\gamma \geq 0$$

Define a function $g(x)$, which is a lower bound for λ:

$$x^2 + \lambda x^\gamma \geq 2\alpha x \Rightarrow \lambda x^\gamma \geq 2\alpha x - x^2 \Rightarrow \lambda \geq \frac{2\alpha x - x^2}{x^\gamma} := g(x)$$

Maximize $g(x)$:

$$\begin{aligned}
g'(x) &= \frac{(2\alpha - 2x)x^\gamma - \gamma x^{1-\gamma}(2\alpha x - x^2)}{(x^\gamma)^2} = \frac{2\alpha - 2x - \gamma x^{-1}(2\alpha x - x^2)}{x^\gamma} = \frac{2\alpha - 2x - 2\alpha\gamma + \gamma x}{x^\gamma} \\
&\Rightarrow \frac{2\alpha - 2x - 2\alpha\gamma + \gamma x}{x^\gamma} = 0 \\
&\Rightarrow 2\alpha - 2x - 2\alpha\gamma + \gamma x = 0 \\
&\quad -2x + \gamma x = 2\alpha\gamma - 2\alpha \\
&x(\gamma - 2) = 2\alpha(\gamma - 1) \\
&x_{\max} = \arg\max_x g(x) = \frac{2\alpha(\gamma-1)}{\gamma - 2} = \frac{-2\alpha(1-\gamma)}{-(2-\gamma)} = \frac{2\alpha(1-\gamma)}{2-\gamma}
\end{aligned}$$

$\lim_{x\to 0^+} g(x) = 0, \quad \lim_{x\to\infty} g(x) = -\infty.$, so $x_{\max}$ maximizes $g(x)$.
Evaluate $g(x_{max})$ to define lower bound for λ:

$$\begin{aligned}
g(x_{\max}) &= \frac{2\alpha x_{\max} - x_{\max}^2}{x_{\max}^\gamma} = x_{\max}^{-\gamma}\left(2\alpha x_{\max} - x_{\max}^2\right) = 2\alpha x_{\max}^{1-\gamma} - x_{\max}^{2-\gamma} = x_{\max}^{1-\gamma}(2\alpha - x_{\max}) \\
&= \left[\frac{2\alpha(1-\gamma)}{2-\gamma}\right]^{1-\gamma} \cdot \left(2\alpha - \frac{2\alpha(1-\gamma)}{2-\gamma}\right) \\
&= \left[\frac{2\alpha(1-\gamma)}{2-\gamma}\right]^{1-\gamma} \cdot \left(\frac{2\alpha(2-\gamma)}{2-\gamma} - \frac{2\alpha(1-\gamma)}{2-\gamma}\right) \\
&= \left[\frac{2\alpha(1-\gamma)}{2-\gamma}\right]^{1-\gamma} \cdot \left(\frac{4\alpha - 2\alpha\gamma - 2\alpha + 2\alpha\gamma}{2-\gamma}\right) \\
&= \left[\frac{2\alpha(1-\gamma)}{2-\gamma}\right]^{1-\gamma} \cdot \frac{2\alpha}{2-\gamma} \\
&= \left[\frac{2\alpha - 2\alpha\gamma}{2-\gamma}\right]^{1-\gamma} \frac{2\alpha}{2-\gamma} = \alpha^{\gamma-1}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma} \alpha\left(\frac{2}{2-\gamma}\right)
\end{aligned}$$

$$g(x_{\max}) = \alpha^{\gamma-1}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma}\alpha\left(\frac{2}{2-\gamma}\right) = \frac{2}{2-\gamma}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma}\alpha^{2-\gamma} = \frac{2}{2-\gamma}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma}|\alpha|^{2-\gamma},$$

since $f(x)$ is a symmetric.
Therefore λ is sufficiently large if it satisfies the following bound:

$$\lambda \geq g(x_{\max}) = \frac{2}{2-\gamma}\left[\frac{2-2\gamma}{2-\gamma}\right]^{1-\gamma}|\alpha|^{2-\gamma}$$

(b) Comparing two iterative algorithms for computing x^*

i. Set $x_0 = \alpha$ and define:

$$x_k = \alpha - \frac{\lambda\gamma}{2}\cdot\frac{|x_{k-1}|^\gamma}{x_{k-1}}, k = 1, 2, 3, \ldots$$

```python
# Q2: ITERATIVE OPTIMIZATION METHODS
import numpy as np


# (b) implement and compare two iterative algoirthms for computing x*
# ---------------------------------------------------------------------------

# part i: iterative algorithm w/ update rule defined on handout
def iterative_alg(alpha, gamma, lamb, tolerance=1e-15, max_iter=5000):
    """
    Iterative root-finding algorithm with update rule:
        x_k = alpha - (lambda * gamma)/2 • (|x_{k-1}|^gamma)/x_{k-1}
    """
    x_0 = alpha  # set x_0=alpha
    converged = False
    x_history = [x_0]
    objective_history = [f_prime(x=x_0, alpha=alpha, gamma=gamma, lamb=lamb)]
    x_prev, k = x_0, 0
    while k < max_iter and not converged:
        # use update rule defined on a3 handout
        x_k = alpha - 0.5 * (lamb * gamma) * ((np.abs(x_prev)) ** gamma) / x_prev
        objective_k = f_prime(x=x_k, alpha=alpha, gamma=gamma, lamb=lamb)
        if np.abs(objective_k) < tolerance:
            converged = True
        x_history.append(x_k)
        objective_history.append(objective_k)
        x_prev = x_k
        k += 1
    return x_history, objective_history, k


```

ii. Newton-Raphson with $x_0 = \alpha$

```python
# part ii: Newton-Raphson algorithm using initial condition x_0=alpha

# helpers to compute f'(x) and f"(x)
def f_prime(x, alpha, gamma, lamb):
    if x == 0:
        raise ValueError("f(x) is not differentiable at cusp x = 0")
    else:
        return 2 * x - 2 * alpha + lamb * gamma * np.abs(x)**(gamma-1) * np.sign(x)


def f_double_prime(x, alpha, gamma, lamb):
    if x == 0:
        raise ValueError("cannot evaluate second derivative at cusp x=0")
    else:
        second_deriv = 2 + lamb * gamma * (gamma - 1) * np.abs(x)**(gamma-2)
        # I got the next 3 lines from chatgpt to prevent
        # "RuntimeWarning: overflow encountered in scalar divide"
        if second_deriv == 0:
            return 1e-6
        return second_deriv if np.abs(second_deriv) > 1e-6 else np.sign(second_deriv) * 1e-6
```

```python


def newton_raphson(alpha, gamma, lamb, tolerance=1e-15, max_iter=5000):
    """
    The Newton-Raphson algorithm using initial condition x_0=alpha
    """
    x_0 = alpha
    x_history = [x_0]
    objective_history = [f_prime(x=x_0, alpha=alpha, gamma=gamma, lamb=lamb)]
    converged = False
    x_prev, k = x_0, 0
    while k < max_iter and not converged:
        # perform Newton-Raphson update
        derivative = objective_history[-1]
        second_derivative = f_double_prime(x=x_prev, alpha=alpha, gamma=gamma, lamb=lamb)
        x_k = x_prev - derivative / second_derivative
        if np.abs(x_k) < 1e-6:  # numerically avoiding x=0 (I used chatgpt for this line)
            x_k = 1e-6 if x_k == 0 else np.sign(x_k) * 1e-6
        objective_k = f_prime(x=x_k, alpha=alpha, gamma=gamma, lamb=lamb)
        if np.abs(objective_k) < tolerance:
            converged = True
        x_history.append(x_k)
        objective_history.append(objective_k)
        x_prev = x_k
        k += 1
    return x_history, objective_history, k


```

Using different values of α, γ, and λ to test these algorithms & determine which one is faster:

```python
# use different values of (alpha, gamma, lamb) to test these algorithms &
#  determine which one is faster

# i asked chat gpt to give me
# different sets of (alpha, gamma, lambda) values to violate the constraint from part (a)

different_values = [(1, 0.5, 1), (2, 0.5, 1), (0.5, 0.8, 0.3), (5, 0.3, 0.5)]

for values in different_values:
    a, g, l = values[0], values[1], values[2]
    iter = iterative_alg(alpha=a, gamma=g, lamb=l)
    nr = newton_raphson(alpha=a, gamma=g, lamb=l)
    # print out the number of iterations required
    print(f"alpha={a}, gamma={g}, lambda={l}")
    print(f"number of iterations required for convergence:")
    print(f"iterative: {iter[2]}, newton-raphson: {nr[2]}")
    print(" ")
```

```
alpha=1, gamma=0.5, lambda=1
number of iterations required for convergence:
iterative: 22, newton-raphson: 4

alpha=2, gamma=0.5, lambda=1
number of iterations required for convergence:
iterative: 12, newton-raphson: 3

alpha=0.5, gamma=0.8, lambda=0.3
number of iterations required for convergence:
iterative: 14, newton-raphson: 4

alpha=5, gamma=0.3, lambda=0.5
number of iterations required for convergence:
iterative: 6, newton-raphson: 2
```

Figure 4: Output from the code chunk above, showing the number of iterations required for the iterative algorithm from part i. and the Newton-Raphson algorithm from part ii. to converge.

As shown in figure 4, the Newton-Raphson algorithm is faster. The iterative algorithm from part i consistently took 3-4 times more iterations to converge, compared to the Newton-Raphson algorithm from part ii.

(c) How the algorithms in part (b) could be used to define a co-ordinate descent algorithm that iteratively finds $\boldsymbol{\beta}^*$ minimizing the objective (bridge estimation loss function):

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^{N} (y^{(i)} - (\boldsymbol{x}^{(i)})^\top \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^{d} |\beta_j|^\gamma$$

with coefficients $\boldsymbol{\beta} \in \mathbb{R}^d$, given data $y^{(i)} \in \mathbb{R}$ and $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$ for $i = 1, \dots, N$ and $\gamma > 0$, $\lambda > 0$ bridge estimates.

i. The first algorithm from part (b) could be used to define a co-ordinate descent algorithm to find $\boldsymbol{\beta}^*$ to minimize the objective if I use initial condition $\boldsymbol{\beta}^{(0)} = \alpha$ and apply the same update rule to update one co-ordinate of $\boldsymbol{\beta} \in \mathbb{R}^d$ at a time, at every iteration:

$$\hat{\beta}_j^{(k)} = \alpha_j - \frac{\lambda\gamma}{2} \cdot \frac{|\hat{\beta}_j^{(k-1)}|^\gamma}{\hat{\beta}_j^{(k-1)}}, \qquad \text{for } j = 1, \dots, d$$

Note: it's not clear to me that this is a true co-ordinate descent algorithm because it doesn't update β_j given fixed values of the other co-ordinates, $\boldsymbol{\beta}{-}j$, but my interpretation of this question was to describe how to use the same algorithm and update rule from part (b) for this problem, and this one didn't use derivatives in part (b).

ii. Newton-Raphson with initial condition $\boldsymbol{\beta}^{(0)} = \alpha$, applying an update rule to one co-ordinate at a time during each iteration.

$$s(\beta_1, \dots, \beta_d) = \frac{\partial}{\partial \beta_j} \mathcal{L}(\boldsymbol{\beta}) = \begin{cases} \sum_{i=1}^{N} -2x_j^{(i)}(y^{(i)} - (\boldsymbol{x}^{(i)})^\top \boldsymbol{\beta}) + \lambda\gamma \operatorname{sign}(\beta_j)|\beta_j|^{(\gamma-1)} & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$

$$h(\beta_1, \dots, \beta_d) = \frac{\partial^2}{\partial \beta_j^2} \mathcal{L}(\boldsymbol{\beta}) = \begin{cases} \sum_{i=1}^{N} -2(x_j^{(i)})^2(y^{(i)} - (\boldsymbol{x}^{(i)})^\top \boldsymbol{\beta}) + \lambda\gamma(1-\gamma)|\beta_j|^{\gamma-2} & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$

At each iteration ($k = 1, 2, 3, \dots$) update each co-ordinate of $\boldsymbol{\beta}^{(k)} \in \mathbb{R}^d$ as follows:

$$\begin{aligned}
\beta_1^{(k)} &= \beta_1^{(k-1)} - \left[h(\beta_1^{(k-1)}, \dots, \beta_d^{(k-1)})\right]^{-1} s(\beta_1^{(k-1)}, \dots, \beta_d^{(k-1)}) \\
\beta_2^{(k)} &= \beta_2^{(k-1)} - \left[h(\beta_1^{(k)}, \beta_2^{(k-1)}, \dots, \beta_d^{(k-1)})\right]^{-1} s(\beta_1^{(k)}, \beta_2^{(k-1)}, \dots, \beta_d^{(k-1)}) \\
&\vdots \\
\beta_j^{(k)} &= \beta_j^{(k-1)} - \left[h(\beta_1^{(k)}, \dots, \beta_{j-1}^{(k)}, \beta_j^{(k-1)}, \dots, \beta_d^{(k-1)})\right]^{-1} s(\beta_1^{(k)}, \dots, \beta_{j-1}^{(k)}, \beta_j^{(k-1)}, \dots, \beta_d^{(k-1)}) \\
&\vdots \\
\beta_d^{(k)} &= \beta_d^{(k-1)} - \left[h(\beta_1^{(k)}, \dots, \beta_{d-1}^{(k)}, \beta_d^{(k-1)})\right]^{-1} s(\beta_1^{(k)}, \dots, \beta_{d-1}^{(k)}, \beta_d^{(k-1)})
\end{aligned}$$

The objective function $\mathcal{L}(\beta)$ is not differentiable at $\beta_j = 0$, similar to how $f(x)$ is not differentiable at its cusp $x = 0$ in part (b). Similarly to the Newton-Raphson algorithm that I implemented in part (b), I can define the first and second partial derivatives with respect to β_j, numerically avoid an both update that goes to $\beta_j = 0$ and $\frac{\partial^2 \mathcal{L}(\beta)}{\partial \beta_j^2}$ in an implementation.

3. **Gradient Descent with Step Size**

The Rosenbrock function (aka banana function):

$$f(x) = (1 - x_1)^2 + (x_2 - x_1^2)^2$$

(a) Compute the gradient and Hessian of the Rosenbrock function:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} -2(1 - x_1) - 4x_1(x_2 - x_1^2) \\ 2(x_2 - x_1^2) \end{pmatrix} = \begin{pmatrix} 4x_1^3 - 4x_1x_2 + 2x_1 - 2 \\ 2x_2 - 2x_1^2 \end{pmatrix}$$

$$\begin{aligned} \nabla^2 f(x) &= \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 x_2} \\ \frac{\partial^2 f}{\partial x_2 x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{pmatrix} \\ &= \begin{pmatrix} \frac{\partial}{\partial x_1}(4x_1^3 - 4x_1x_2 + 2x_1 - 2) & \frac{\partial}{\partial x_2}(4x_1^3 - 4x_1x_2 + 2x_1 - 2) \\ \frac{\partial}{\partial x_1}(2x_2 - 2x_1^2) & \frac{\partial}{\partial x_2}(2x_2 - 2x_1^2) \end{pmatrix} \\ &= \begin{pmatrix} 12x_1^2 - 4x_2 + 2 & -4x_1 \\ -4x_1 & 2 \end{pmatrix} \end{aligned}$$

Python implementation of a function that given $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$ returns $f(x)$, the gradient $\nabla f(x) \in \mathbb{R}^2$, and the Hessian $\nabla^2 f(x) \in \mathbb{R}^{2\times 2}$:

```python
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Q3: GRADIENT DESCENT WITH STEP SIZE
5  # -------------------------------------------
6
7
8  # (a) implement a function that given x returns f(x), the gradient, and the hessian
9
10 def rosenbrock(x):
11     """
12     Given a 2x1 vector x, compute the value of the Rosenbrock function f(x) = (1 - x1)^2 + (x2 - x1^2)^2,
13     its gradient vector f'(x), and its hessian matrix f''(x)
14     """
15     x_1, x_2 = x[0], x[1]
16     # compute f(x)
17     f_x = (1 - x_1) ** 2 + (x_2 - x_1 ** 2) ** 2
18     # compute the gradient (2x1 vector)
19     grad = np.array([-2 * (1 - x_1) - 4 * x_1 * (x_2 - x_1 ** 2),
20                      2 * (x_2 - x_1 ** 2)])
21     # compute the Hessian (2x2 matrix)
22     hess = np.array([
23         [12 * x_1 ** 2 - 4 * x_2 + 2, -4 * x_1],
24         [-4 * x_1, 2]
25     ])
26     return f_x, grad, hess
27
28
```

Line 8 of the code chunk above computes output of the Rosenbrock function, $f(x) = (1 - x_1)^2 + (x_2 - x_1^2)^2$.

Lines 10-11 compute the gradient vector, $\nabla f(x) = \begin{pmatrix} 4x_1^3 - 4x_1x_2 + 2x_1 - 2 \\ 2x_2 - 2x_1^2 \end{pmatrix}$.

Lines 13-16 compute the Hessian matrix, $\nabla^2 f(x) = \begin{pmatrix} 12x_1^2 - 4x_2 + 2 & -4x_1 \\ -4x_1 & 2 \end{pmatrix}$.

Lastly, line 17 returns 1) the function value $f(x) \in \mathbb{R}$, the gradient $\nabla f(x) \in \mathbb{R}^2$, and the Hessian $\nabla^2 f(x) \in \mathbb{R}^{2\times 2}$

(b) Gradient descent python implementation:

```python
# (b) gradient descent implementation
def rosenbrock_gradient_descent(x_0=np.array([-1.5, 1.1]), alpha=0.05, epsilon=1e-4, n_max=300):
    """
    Perform gradient descent with the following update rule at each iteration:
        x_k+1 = x_k -  • f(x_k)

    Terminate when || f(x_k) || <  or maximum number of iterations, nmax is reached.

    Return:
        - optimization path {x_k}
        - convergence history {f(x_k)}
    """
    optimization_path = [x_0.copy()]
    convergence_history = []
    x_k = x_0.copy()
    for k in range(n_max):
        f_x_k, grad_x_k, _ = rosenbrock(x_k)
        convergence_history.append(f_x_k.copy())
        # stop early if || f(x_k) ||_2 <  (I think this denotes L2 norm)
        if np.linalg.norm(grad_x_k) < epsilon:
            return optimization_path, convergence_history
        # otherwise perform a gradient descent update x_k+1 = x_k -  • f(x_k)
        x_k_1 = x_k - alpha * grad_x_k
        optimization_path.append(x_k_1.copy())
        x_k = x_k_1
    return optimization_path, convergence_history


```

Choosing α such that the method converges in fewer than 300 iterations by testing out different values of α between 0.25 and 0.0001 and observing the optimization path and convergence history:

```python
def plot_rosenbrock_convergence_history(conv_hist, alpha, epsilon):
    """
    Plot the Rosenbrock Function gradient descent convergence history.
    Shows the value of the objective function, the L2 norm of the gradient
    vector f'(x) after every gradient descent iteration.
    """
    plt.figure(figsize=(8, 5))
    plt.plot(conv_hist, linewidth=2)
    plt.title(
        rf"Convergence History "
        rf"for $\alpha={alpha}$, $\epsilon={epsilon}$"
    )
    plt.xlabel("Iteration")
    plt.ylabel(r"Objective value, $\|\nabla f(x_k) \|_2$")
    plt.grid(True)
    plt.tight_layout()
    plt.show()


def plot_rosenbrock_optimization_path(opt_path, alpha, epsilon):
    """
    Plot the Rosenbrock function contours & gradient descent optimization path.
    Disclosure of generative AI use: I used Copilot to produce this function. I asked it to
    help me plot the Rosenbrock function & optimization path.
    """
    path = np.array(opt_path)
    x1s, x2s = path[:, 0], path[:, 1]
    # Create a proper grid for contour plotting
    X1 = np.linspace(-2, 2, 400)
    X2 = np.linspace(-1, 3, 400)
    XX, YY = np.meshgrid(X1, X2)
```

```
88      # Rosenbrock function for grid evaluation
89      ZZ = (1 - XX)**2 + 100 * (YY - XX**2)**2
90      plt.figure(figsize=(10, 7))
91      # Log-spaced contour levels reveal the valley shape
92      levels = np.logspace(-1, 3, 20)
93      plt.contour(XX, YY, ZZ, levels=levels, cmap="viridis")
94      # Optimization path
95      plt.plot(x1s, x2s, "r.-", markersize=8, linewidth=1.5, label="Optimization path")
96      # Start and end markers
97      plt.plot(x1s[0], x2s[0], "go", label="Start")
98      plt.plot(x1s[-1], x2s[-1], "bo", label="End")
99      plt.title(
100         rf"Optimization Path "
101         rf"for $\alpha={alpha}$, $\epsilon={epsilon}$"
102     )
103     plt.xlabel("x1")
104     plt.ylabel("x2")
105     plt.legend()
106     plt.grid(True)
107     plt.show()
108
109
110 # try out different values of alpha, using threshold eps=0.0003 like in lecture
111 learning_rates = [0.25, 0.1, 0.01, 0.001, 0.0001]
112 epsilons = [0.0003]
113 for lr in learning_rates:
114     for eps in epsilons:
115         rosenbrock_result = rosenbrock_gradient_descent(alpha=lr, epsilon=eps)
116         plot_rosenbrock_optimization_path(opt_path=rosenbrock_result[0], alpha=lr, epsilon=eps)
117         plot_rosenbrock_convergence_history(conv_hist=rosenbrock_result[1], alpha=lr, epsilon=eps)
118
119
```

To determine a fixed step size α, I plotted compare the optimization path & convergence history over a maximum of 300 iterations for $\alpha = 0.25, 0.1, 0.01, 0.001, 0.0001$. The the output of lines 111-117 in the code chunk above are shown on the next page.

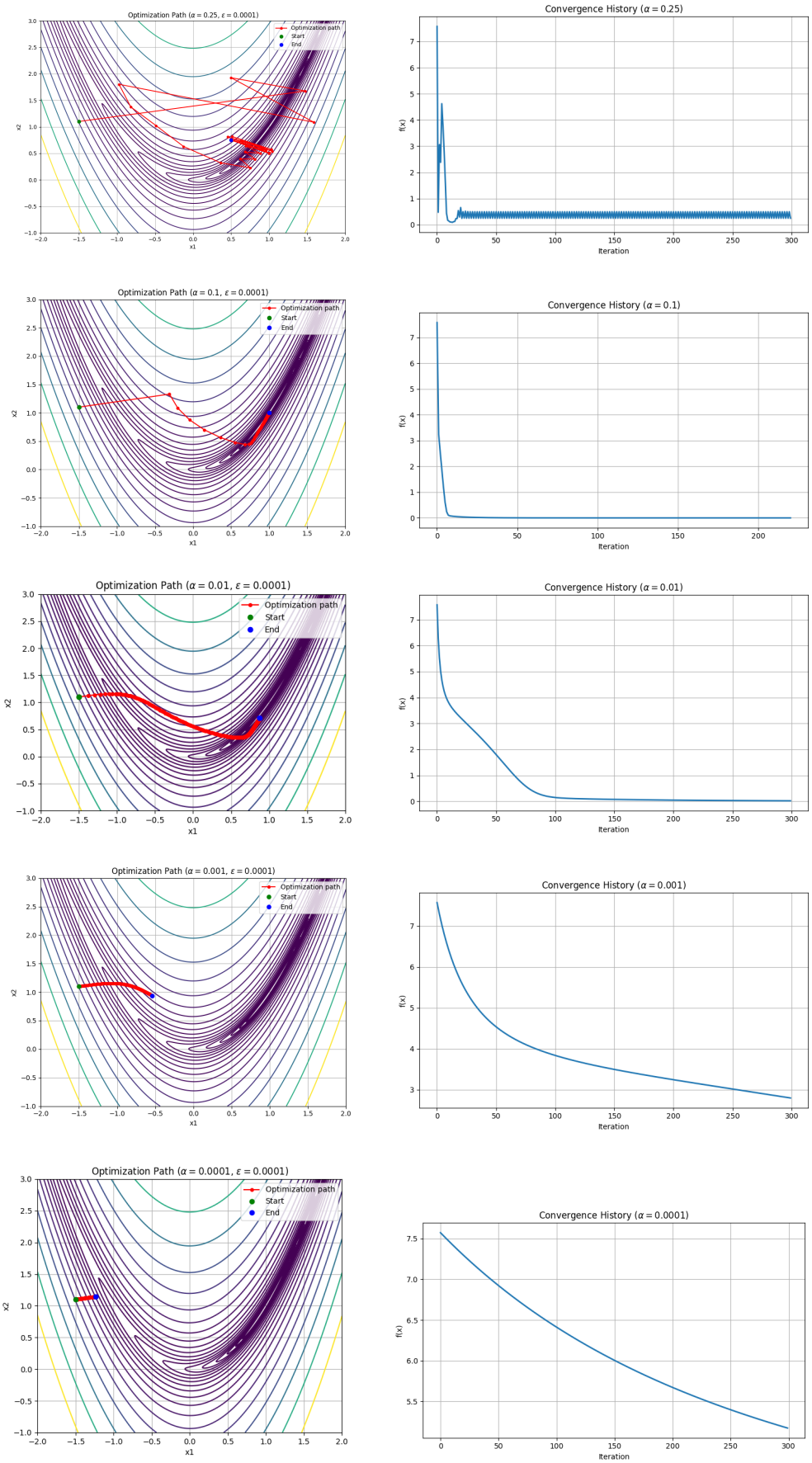

Figure 5: Optimization path & convergence history of gradient descent using $\alpha \in [0.25, 0.1, 0.01, 0.001, 0.0001]$

.

Figure 4 shows the optimization path and convergence history of rosenbrock function gradient descent using fixed learning rate $\alpha = [0.25, 0.1, 0.01, 0.001, 0.0001]$.

Based on these results, I selected $\alpha = 0.01$ x_k, because x_k converges to $\begin{pmatrix} x_1^* \\ x_2^* \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $f(x)$ converges to 0 after less than 200 iterations of gradient descent.

Additionally, we can see optimization path diverges & the value of the objective function starts oscillating after approximately 25 iterations of gradient descent, with $\alpha = 0.25$, indicating that that learning rate was too large. Additionally, convergence takes longer with $\alpha = 0.01$, the optimization path does not converge to $\begin{pmatrix} x_1^* \\ x_2^* \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and the value of the objective function does not converge to 0 at all within 300 iterations, when step sizes $\alpha = 0.001$ and $\alpha = 0.0001$ were used, indicating that they were too small.

If I choose $\alpha = 0.1$ so that the method converges within less than 300 iterations, it takes around 50 iterations of gd to reach the lowest contour of the rosenbrock valley, where convergence slows down considerably. Convergence slows down in this region because it's narrow & the hessian becomes ill conditioned (I googled this).

(c) Implementing an improved gradient descent method with adaptive step size Descent direction:

$$u_k = -\frac{\nabla f(x_k)}{||\nabla f(x_k)||}$$

One-dimensional function:

$$\phi(a) = f(x_k + \alpha u_k) = f\left(x_k - \alpha - \frac{\nabla f(x_k)}{||\nabla f(x_k)||}\right)$$

Objective function:

$$\phi'(\alpha) = \frac{\partial}{\partial \alpha} f(x_k + \alpha u_k) = \nabla f(x_k + \alpha u_k)^\top u_k$$

$\phi'(\alpha)$ implementation:

```python
# (c) gradient descent improvement
# ---------------------------------

def phi_prime(alpha, x, u):
    """
    Computes phi'(alpha)
    """
    gradient = rosenbrock(x + alpha * u)[1]
    return np.dot(gradient.T, u)


```

Bisection method helper (to be used to compute α_k in each gradient descent iteration):

```python
def bisection_alpha(x, u, alpha_lb, alpha_ub, tau):
    """
    Bisection method to find a root for objective function '()
    Statement on gen ai use: I used ChatGPT to debug this function.
    """
    phi_prime_lb = phi_prime(alpha_lb, x, u)
    phi_prime_ub = phi_prime(alpha_ub, x, u)
    count = 0
    # expanding the interval until it captures a root
    while phi_prime_lb * phi_prime_ub > 0 and count < 50:  # I got this suggestion from ChatGPT
        alpha_ub *= 2
        phi_prime_ub = phi_prime(alpha_ub, x, u)
        count += 1
    if (phi_prime_lb * phi_prime_ub) > 0:  # not able to capture a root after 50 iterations
        raise ValueError("Failed to capture a root within the interval")
    while True:
        alpha_mid = alpha_mid = 0.5 * (alpha_lb + alpha_ub)
```

```python
        phi_prime_mid = phi_prime(alpha_mid, x, u)
        if abs(phi_prime_mid) < tau:
            return alpha_mid
        if phi_prime_lb * phi_prime_mid < 0:  # sign(phi_prime_lb])  sign(phi_prime_mid)
            alpha_ub = alpha_mid
            phi_prime_ub = phi_prime_mid
        else:  # sign(phi_prime_mid)  sign(phi_prime_ub)
            alpha_lb = alpha_mid
            phi_prime_lb = phi_prime_mid


```

Improved gradient descent method implementation:

```python
def improved_rosenbrock_gradient_descent(x_0=np.array([-1.2, 1.1]), nmax=300, epsilon=1e-4):
    """
    Perform an improved gradient descent method with update rule:
        x_k+1 = x_l - alpha_k • f(x_k)
    where step size alpha_k is selected using a bisection line search at
    each iteration.

    Return:
        - optimization path {x_k}
        - convergence history {f(x_k)}
    """
    opt_path = [x_0.copy()]
    conv_hist = []
    x_k = x_0.copy()
    for k in range(nmax):
        f_x_k, grad_x_k, hess_x_k = rosenbrock(x_k)
        conv_hist.append(f_x_k.copy())
        # stop early if || f(x_k) ||_2 <  (I think this denotes L2 norm)
        if np.linalg.norm(grad_x_k) < epsilon:
            return opt_path, conv_hist
        # compute the descent direction u_k
        u_k = -1 * (grad_x_k / np.linalg.norm(grad_x_k, ord=2))
        # compute the step size at the kth iteration
        # compute alpha_ub
        hess_eigvals = np.linalg.eigvals(hess_x_k)
        lambda_1_x = np.max(hess_eigvals)
        # upper bound for alpha: 2 / lambda_1_x (lecture 8)
        alpha_k = bisection_alpha(x=x_k, u=u_k, alpha_lb=0,
                                  alpha_ub=(2 / lambda_1_x), tau=0.0001)
        # perform gradient descent update
        # x_k_1 = x_k - alpha_k * grad_x_k
        x_k_1 = x_k + alpha_k * u_k
        opt_path.append(x_k_1.copy())
        x_k = x_k_1
    return opt_path, conv_hist


```

In my `improved_rosenbrock_gradient_descent` implementation, I determine $\alpha_{UB}^{(k)} \leq \frac{2}{\lambda_1(x^{(k)})}$ using the bound from lecture 8 slide 6, where $\lambda_1(x^{(k)})$ is the largest eigenvalue of the Hessian, $\nabla^2 f(x)$. In my `bisection_alpha` helper function, I double my interval if $\text{sign}(\phi'(0)) = \text{sign}(\phi'(\alpha_{UB}))$ until the signs become opposite to ensure that it contains a root (I got this suggestion from ChatGPT and office hours).
Plotting the convergence history & optimization path of this improved gradient descent method with adaptive step size via bisection line search:

```python
# perform gd w/ adaptive step size
improved_result = improved_rosenbrock_gradient_descent()

# plot the optimization path (i used chat gpt to get a nice plot for this)
path = np.array(improved_result[0])
x1s, x2s = path[:, 0], path[:, 1]
X1 = np.linspace(-2, 2, 400)
```

```python
X2 = np.linspace(-1, 3, 400)
XX, YY = np.meshgrid(X1, X2)
ZZ = (1 - XX) ** 2 + 100 * (YY - XX ** 2) ** 2
plt.figure(figsize=(10, 7))
levels = np.logspace(-1, 3, 20)
plt.contour(XX, YY, ZZ, levels=levels, cmap="viridis")
plt.plot(x1s, x2s, "r.-", markersize=8, linewidth=1.5, label="Optimization path")
plt.plot(x1s[0], x2s[0], "go", label="Start")
plt.plot(x1s[-1], x2s[-1], "bo", label="End")
plt.title(
    rf"Optimization path of gradient descent using adaptive step size "
    rf"($\epsilon={1e-4}$)"
)
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.grid(True)
plt.show()

# plot the convergence history
plt.figure(figsize=(8, 5))
plt.plot(improved_result[1], linewidth=2)
plt.title(
    rf"Convergence history of gradient descent using adaptive step size "
    rf"($\epsilon={1e-4}$)"
)
plt.xlabel("Iteration")
plt.ylabel(r"f(x)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

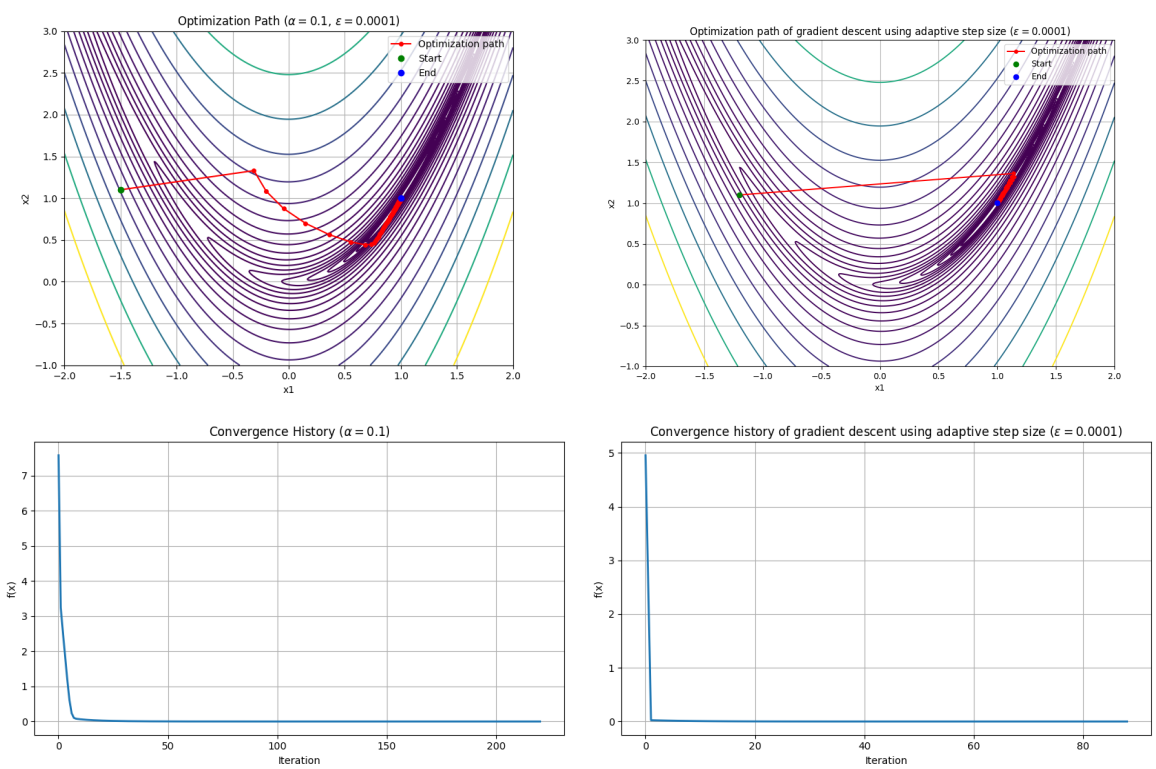

Figure 6: Optimization path & convergence history of Rosenbrock Function gradient descent with fixed step size ($\alpha = 0.1$) from part (b) vs improved gradient descent method using a bisection line search adaptive step size

Comparing the optimization paths and convergence histories of gradient descent with a fixed step size of $\alpha = 0.1$ vs an adaptive step size: 1) the main difference in the optimization paths that I notice is that the one with a fixed step size $\alpha = 0.1$ takes more, smaller steps before reaching the lowest contour of the rosenbrock valley, whereas the one with adaptive step size appears to take one very large step and reach the lowest contour of the rosenbrock valley immediately, and then takes much smaller steps right away once reaching that region. 2) Consequently, the gradient descent method using adaptive step size converges faster (within 90 iterations) than the gradient descent method with the fixed step size $\alpha = 0.1$, which took just over 200 (but still less than 300) iterations to converge. 3) The bisection method requires the following computations at each iteration: u_k, which involves the L2 norm of $x_k \in \mathbb{R}^d$, which costs $\mathcal{O}(\lceil)$, finding the largest eigenvalue of the Hessian matrix which costs $\mathcal{O}(d^3)$, and $\phi'(\alpha) = \nabla f(x + \alpha u)^\top u$, each time we half the interval, which costs $\mathcal{O}(d)$ computations since it's the dot product of two $d \times 1$ vectors. Therefore, bisection method requires $\mathcal{O}(d^3)$ additional computations, but it can converge after significantly fewer iterations.

4. **EM Algorithm for a Mixture of Exponentials**
Let $x^1, \dots x^N$ be independent observations from a two-component mixture of exponential distributions:

$$p(x; \lambda_1, \lambda_2, p) = \theta \cdot \lambda_1 e^{-\lambda_1 x} + (1-\theta) \cdot \lambda_2 e^{-\lambda_2 x}, x \geq 0$$

where $\lambda_1, \lambda_2 > 0$ and $0 < \theta < 1$ are unknown parameters

(a) The complete data is defined as $Y^i = (X^i, \Delta_i)$, where X^i is the observed value and $\Delta_i \sim \text{Bernoulli}(\theta)$ is equal to 1 if X^i comes from component 1, and 0 if X^i comes from component 2.

The complete data likelihood and log-likelihood are as follows:

$$\mathcal{L}_c(\lambda_1, \lambda_2, \theta) = \prod_{i=1}^{n} \left[\theta \lambda_1 e^{-\lambda_1 x_i}\right]^{\Delta_i} \cdot \left[(1-\theta)\lambda_2 e^{-\lambda_2 x_i}\right]^{1-\Delta_i}$$

$$\begin{aligned}
\log \mathcal{L}_c(\lambda_1, \lambda_2, \theta) &= \sum_{i=1}^{N} \log \left\{ \left[\theta \lambda_1 e^{-\lambda_1 x_i}\right]^{\Delta_i} \cdot \left[(1-\theta)\lambda_2 e^{-\lambda_2 x_i}\right]^{1-\Delta_i} \right\} \\
&= \sum_{i=1}^{N} \Delta_i \log\left(\theta \lambda_1 e^{-\lambda_1 x_i}\right) + (1-\Delta_i) \log\left((1-\theta)\lambda_2 e^{-\lambda_2 x_i}\right) \\
&= \sum_{i=1}^{N} \Delta_i \left[\log\theta + \log\lambda_1 - \lambda_1 x_i\right] + (1-\Delta_i)\left[\log(1-\theta) + \log\lambda_2 - \lambda_2 x_i\right] \\
&= \sum_{i=1}^{N} \Delta_i \left[\log\theta + \log\lambda_1 - \lambda_1 x_i\right] + \sum_{i=1}^{N} (1-\Delta_i)\left[\log(1-\theta) + \log\lambda_2 - \lambda_2 x_i\right] \\
&= \left(\sum_{i=1}^{N} \Delta_i\right)\left[\log\theta + \log\lambda_1\right] - \lambda_1 \sum_{i=1}^{N} \Delta_i x_i + \left(N - \sum_{i=1}^{N} \Delta_i\right)\left[\log(1-\theta) + \log\lambda_2\right] - \lambda_2 \sum_{i=1}^{N} (1-\Delta_i) x_i
\end{aligned}$$

(b) E-step derivation:

$$\begin{aligned}
\mathbb{E}[\Delta_i | X^i = x^i] &= 1 \cdot \mathbb{P}[\Delta_i = 1 | X^i = x^i] + 0 \cdot \mathbb{P}[\Delta_i = 0 | X^i = x^i] \\
&= \mathbb{P}[\Delta_i = 1 | X^i = x^i] \\
&= \frac{\theta \lambda_1 e^{-\lambda_1 x_i}}{\theta \lambda_1 e^{-\lambda_1 x_i} + (1-\theta)\lambda_2 e^{-\lambda_2 x_i}} \\
&:= \hat{\delta}_i(\lambda_1, \lambda_2, \theta)
\end{aligned}$$

Expected data log likelihood for the EM update:

$$\begin{aligned}
\log \mathcal{L}_k(\lambda_1, \lambda_2, \theta) &= \mathbb{E}\Big[\log \mathcal{L}_c | X^i = x^i,\ i = 1, ..., N\Big] \\
&= \sum_{i=1}^{N}\sum_{r=1}^{2} \delta_{ir}(\log \pi_r + \log p_r(x_i; \lambda_r) \\
&= \sum_{r=1}^{2}\Big\{\delta_{ir}\Big\}\log \pi_r + \sum_{r=1}^{2}\sum_{i=1}^{N} \delta_{ir} \log p_r(x_i, \lambda_r) \\
&\text{substitute the following:} \\
&\pi_r = \begin{cases} \hat{\theta}, & \text{if } r = 1 \\ (1-\hat{\theta}), & \text{if } r = 2 \end{cases} \\
&\delta_{ir} = \begin{cases} \hat{\delta}_i, & \text{if } r = 1 \\ 1-\hat{\delta}_i, & \text{if } r = 2 \end{cases} \\
&\log p_r(x_i, \lambda_r) = \begin{cases} \log(\lambda_1 e^{-\lambda_1 x_i}) = \log \lambda_1 - \lambda_1 x_i, & \text{if } r = 1 \\ \log(\lambda_2 e^{-\lambda_2 x_i}) = \log \lambda_2 - \lambda_2 x_i, & \text{if } r = 2 \end{cases} \\
&= \left(\sum_{i=1}^{N} \hat{\delta}_i\right)\log \hat{\theta}^{(k)} + \left(\sum_{i=1}^{N}(1-\hat{\delta}_i)\right)\log(1-\hat{\theta}^{(k)}) \\
&\quad + \sum_{i=1}^{N} \hat{\delta}_i(\log \hat{\lambda}_1^{(k)} - \hat{\lambda}_1^{(k)} x_i) + \sum_{i=1}^{N}(1-\hat{\delta}_i)(\log \hat{\lambda}_2 - \hat{\lambda}_2 x_i)
\end{aligned}$$

(c) M-step update equations for $\hat{\lambda}_1, \hat{\lambda}_2, \hat{\theta}$
The update for $\hat{\theta}$ can be computed directly:

$$\hat{\theta}^{(k+1)} = \frac{1}{N}\sum_{i=1}^{N} \hat{\delta}_i(\hat{\lambda}_1^{(k)}, \hat{\lambda}^{(k)}, \hat{\theta}^{(k)})$$

The updated values of $\hat{\lambda}_1, \hat{\lambda}_2$ should maximize $\log \mathcal{L}_k$ (I computed these by deriving $\log \mathcal{L}_k$ wrt each parameter, setting to 0, and solving).

$$\hat{\lambda}_1^{(k+1)} = \frac{\sum_{i=1}^{N} \hat{\delta}_i}{\sum_{i=1}^{N} \hat{\delta}_i x_i}$$

$$\hat{\lambda}_2^{(k+1)} = \frac{\sum_{i=1}^{N}(1-\hat{\delta}_i)}{\sum_{i=1}^{N}(1-\hat{\delta}_i)x_i}$$

(d) Generate $N = 200$ observations using true parameters $\lambda_1 = 1, \lambda_2 = 5, \theta = 0.4$, then implement the EM algorithm in python.

```python
import numpy as np
from numpy import random
import matplotlib.pyplot as plt

rng = random.default_rng(123)

# generate the data
num_obs = 200   # N=200 observations
true_theta, true_lambda1, true_lambda2 = 0.4, 1, 5  # true parameter values
component = rng.binomial(1, true_theta, size=num_obs)
observed_data = np.where(component == 1,
                         rng.exponential(scale=1/true_lambda1, size=num_obs),
                         rng.exponential(scale=1/true_lambda2, size=num_obs)
                         )


```

```
17 def EM(x=observed_data, theta_init=0.5, lamb1_init=2.0,
18        lamb2_init=3.0, tolerance=1e-12):
19     """
20     Expectation maximization algorithm for a mixture of two exponential components.
21     """
22     n = len(x)
23     # initialize & keep track of the log likelihood
24     ll_init = np.sum(np.log(theta_init * lamb1_init * np.exp(- lamb1_init * x) +
25                             (1 - theta_init) * lamb2_init * np.exp(- lamb2_init * x)
26                             )
27                      )
28     log_likelihood_values = [ll_init]
29     theta, lamb1, lamb2, ll = theta_init, lamb1_init, lamb2_init, ll_init
30     converged = False
31     while not converged:  # run algorithm until convergence
32         # E-step: compute E[Delta_i | X_i = x_i]
33         expected_membership = (theta * lamb1 * np.exp(- lamb1 * x)) / \
34                               (theta * lamb1 * np.exp(- lamb1 * x) + (1 - theta) * lamb2 * np.exp(- lamb2 * x))
35         # M-step: compute updated lamb1, lamb2, theta using closed-form equations
36         # to maximize expected log likelihood
37         theta_new = (1 / n) * np.sum(expected_membership)
38         lamb1_new = np.sum(expected_membership) / np.sum(expected_membership * x)
39         lamb2_new = np.sum(1 - expected_membership) / np.sum((1 - expected_membership) * x)
40         # update the observed data log likelihood
41         ll_new = np.sum(np.log(theta_new * lamb1_new * np.exp(- lamb1_new * x) +
42                                (1 - theta_new) * lamb2_new * np.exp(- lamb2_new * x)
43                                ))
44         if np.abs(ll_new - ll) < tolerance:
45             converged = True    # stop if the EM alg has converged
46         # update params + keep track of log likelihood
47         log_likelihood_values.append(ll_new)
48         theta, lamb1, lamb2, ll = theta_new, lamb1_new, lamb2_new, ll_new
49     return lamb1, lamb2, theta, log_likelihood_values
50
51
```

Lines 7-14 of the code chunk shown above generate $N = 200$ observations from a 2-component mixture of exponentials with true parameters $\lambda_1 = 1, \lambda_2 = 5, \theta = 0.4$, and I define those samples as the `observed_data`. Lines 17-49 contain my implementation of the `EM` algorithm. I start by initializing my parameters, and chose an arbitrary starting point of $\lambda_1^{(0)} = 2, \lambda_2^{(0)} = 3, \theta^{(0)} = 0.5$ (to be discussed further in part e), computed the initial observed data log likelihood, then iteratively computed $\mathbb{E}[\Delta_i | X^i = x^i]$, as defined in part (b) in my E-step in lines 32-35 and my updates for $\hat{\theta}, \hat{\lambda_1}, \hat{\lambda_2}$, as defined in part (c) until convergence is reached. Every time I updated the parameters, I kept track of them and re-computed + recorded the value of the log likelihood. My while loop breaks when the algorithm converges and in this implementation, that happened within a reasonable number of iterations, which will be discussed in part (e).

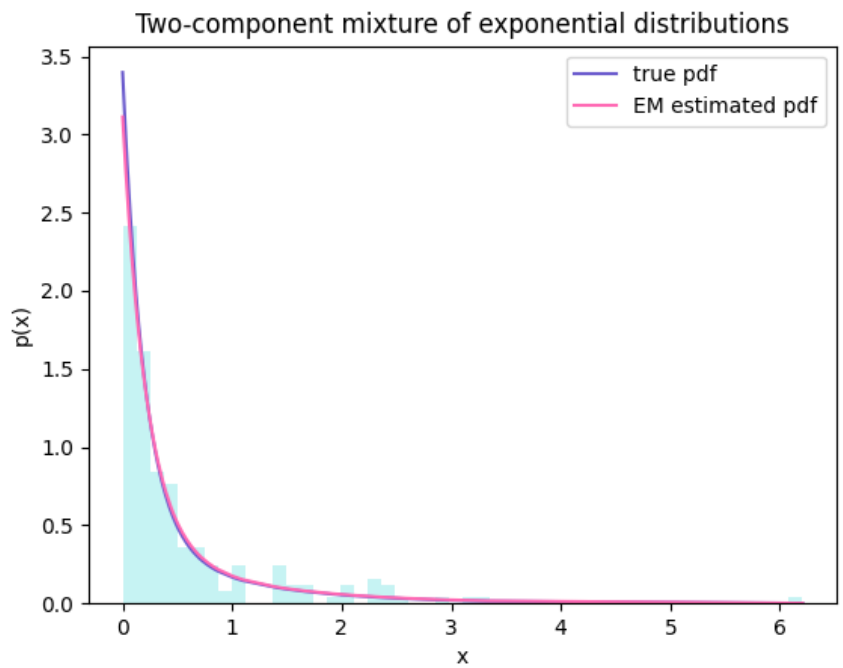

Figure 7: Observed values, $p(x)$ using the true parameters $\lambda_1 = 1, \lambda_2 = 1, \theta = 0.4$, and $p(x)$ the EM-estimated parameters $\hat{\lambda}_1, \hat{\lambda}_2, \hat{\theta}$. The observed values, pdf using true parameters, and pdf using EM-estimated parameters are all pretty close.

.

(e) Plot of the log likelihood to show the monotone increasing property of the EM algorithm & how many iterations were required until convergence.

```
# run the EM algorithm on a 2 component mixture of exponentials
lamb1_hat, lamb2_hat, theta_hat, log_likelihood = EM()

# plot log likelihood at each iteration
plt.figure(figsize=(10, 7))
plt.title("Log Likelihood vs Number of EM Iterations")
plt.xlabel("# iterations")
plt.ylabel("Observed data log likelihood")
plt.plot(log_likelihood, '-o', markersize=2.1, color="magenta")
plt.show()

# determine how many iterations EM required before convergence
num_its_conv = len(log_likelihood)
print(f"Number of EM iterations required before convergence: {num_its_conv}")


```

```
Number of EM iterations required before convergence: 140
```

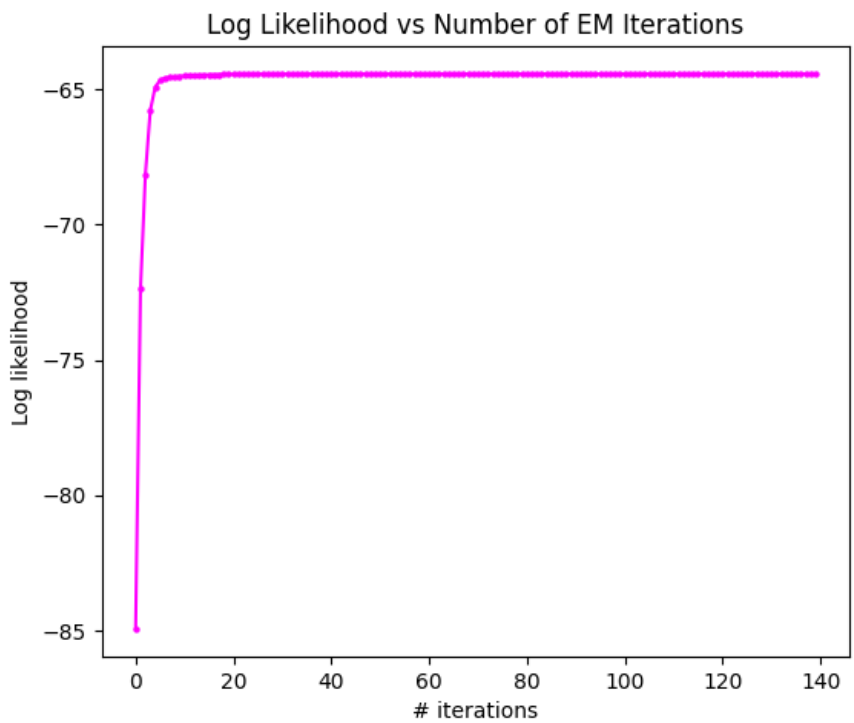

Figure 8: Value of log likelihood vs the number of iterations of the EM algorithm defined in part (d) until convergence. It increases at every iteration, which is consistent with the EM algorithm monotone increasing property.

(f) Try at least 3 different initial values, $(\theta^{(0)}, \lambda_1^{(0)}, \lambda_2^{(0)})$ to determine whether EM runs converge to the same solution.
Is it unimodal? No.

```
# 5 different starting values to try
starting_values = [(0.95, 16, 1), (0.45, 1.5, 4), (0.67, 6, 7),
                   (0.38, 0.98, 5.2), (0.10, 10, 10)]

print("EM solutions after convergence using different starting values\n")
for i, (theta0, l1_0, l2_0) in enumerate(starting_values, start=1):
    result = EM(x=observed_data, theta_init=theta0, lamb1_init=l1_0, lamb2_init=l2_0)
    print(f"Starting values set {i}: lambda1={l1_0}, lambda2={l2_0}, theta={theta0}")
    print(f"  lambda1: {result[0]}")
    print(f"  lambda2: {result[1]}")
    print(f"  theta:   {result[2]}")
    print(f"  log likelihood: {result[3]}\n")
```

```
EM solutions after convergence using different starting values

Starting values: lambda1=16, lambda2=1, theta=0.95
EM solution: lambda1=4.558704442941927, lambda2=0.9482487044495415, theta=0.6001270341724836
log likelihood=-64.43865312362657

Starting values: lambda1=1.5, lambda2=4, theta=0.45
EM solution: lambda1=0.9482487093216088, lambda2=4.558704466677316, theta=0.3998729692423016
log likelihood=-64.43865312362666

Starting values: lambda1=6, lambda2=7, theta=0.67
EM solution: lambda1=0.9482486804385208, lambda2=4.558704325966763, theta=0.39987294899842324
log likelihood=-64.438653123626

Starting values: lambda1=0.98, lambda2=5.2, theta=0.38
EM solution: lambda1=0.948248694361338, lambda2=4.558704393794943, theta=0.3998729587567918
log likelihood=-64.43865312362632

Starting values: lambda1=10, lambda2=10, theta=0.1
EM solution: lambda1=1.8072054990673996, lambda2=1.8072054990674007, theta=0.10000000000000002
log likelihood=-81.64365419339848
```

Figure 9: EM algorithm solutions, $(\hat{\lambda}_1, \hat{\lambda}_2, \hat{\theta})$, after using different five different sets of starting values $(\lambda_1^{(0)}, \lambda_2^{(0)}, \theta^{(0)})$. The EM algorithm appears to converge to the same solution, $(\hat{\lambda}_1 \approx 0.9482, \hat{\lambda}_2 \approx 4.5587, \hat{\theta} \approx 0.999)$ when my starting value for λ_1 is smaller than my starting value for λ_2 and my starting value θ_0 is not too close to 0 or 1. However, this is not always the case. in the first example where I used starting values $\lambda_1^{(0)} = 16 >> \lambda_2^{(0)} = 1, \theta^{(0)} = 0.95$, where $\hat{\theta} \approx 0.6$ and the $\hat{\lambda_1}, \hat{\lambda}_2$ were flipped, or in the last example where I used starting values $\lambda_1^{(0)} = \lambda_2^{(0)} = 10, \theta^{(0)} = 0.1$, which are nowhere close to the true values, so the EM algorithm may have converged to a local maximum too soon, before reaching the region of interest. This shows that the EM algorithm is not guaranteed to always converge to a global maximum, but it is monotone increasing, meaning that the value of its log likelihood will increase with every iteration (Lecture 9 slide 45).

Resources Used

- https://medium.com/tech-spectrum/visualizing-gradient-descent-on-the-rosenbrock-function-2d08f3fa3963
- ChatGPT and copilot for debugging my code & creating visualizations in q3
- Roger Grosse CSC311 course notes
- https://www.math.mcgill.ca/dstephens/557/Handouts/Math557-07-EM.pdf
- https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1056/handouts/fsnlp-em-slides-6.pdf
- Office hours & classmates (I discussed this assignment w/ the instructors, Clement, and Avichai)